

Objective:

TO UNDERSTAND AND PERFORM DATA SHARING BETWEEN COMPUTERS IN A LOCAL AREA NETWORK (LAN).

Apparatus / Requirements:

- Two or more computers connected in a LAN
- Ethernet cables or Wi-Fi connectivity
- Windows/Linux OS
- Shared folder or file
- IP configuration tool (Command Prompt / Terminal)

Theory:

A LAN (Local Area Network) is a network that connects computers within a limited area such as a home, school, or office. Data sharing in LAN is commonly done using file/folder sharing features provided by the OS, and access is usually done via IP addresses or device names.

Procedure:

1. **Connect all computers to the same LAN** using a switch, router, or Ethernet cable.
2. On the **sharing computer**, open **Command Prompt** and run **ipconfig /all** to check the **IPv4 address**.

```
C:\Users\Dell>ipconfig/all

Windows IP Configuration

Host Name . . . . . : DESKTOP-VORK453
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet:

Connection-specific DNS Suffix . . :
Description . . . . . : Realtek PCIe GbE Family Controller
Physical Address. . . . . : 6C-3C-8C-27-AE-4A
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::fee2:a951:8f81:b7c%4(Preferred)
IPv4 Address. . . . . : 192.168.1.82(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Sunday, May 25, 2025 11:32:43 PM
Lease Expires . . . . . : Monday, May 26, 2025 11:32:43 PM
Default Gateway . . . . . : 192.168.1.254
DHCP Server . . . . . : 192.168.1.254
DHCPv6 IAID . . . . . : 74202252
DHCPv6 Client DUID. . . . . : 00-01-00-01-2F-90-8C-C0-6C-3C-8C-27-AE-4A
DNS Servers . . . . . : 202.79.32.4
NetBIOS over Tcpip. . . . . : Enabled
```

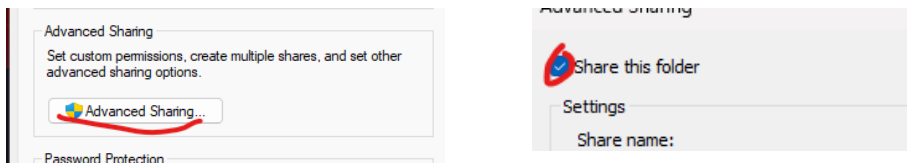
3. On the receiving computer, check if the sharing system is reachable by typing **ping <IP address>** in the Command Prompt.

```
C:\Users\Dell>ping 192.168.1.77

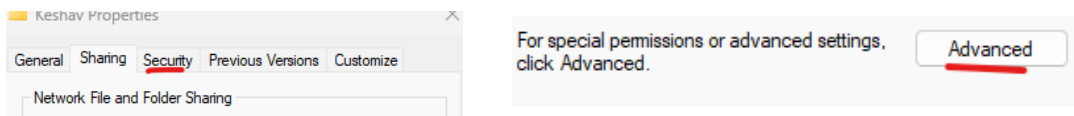
Pinging 192.168.1.77 with 32 bytes of data:
Reply from 192.168.1.77: bytes=32 time=1ms TTL=128
Reply from 192.168.1.77: bytes=32 time=1ms TTL=128
Reply from 192.168.1.77: bytes=32 time=2ms TTL=128
Reply from 192.168.1.77: bytes=32 time=2ms TTL=128

Ping statistics for 192.168.1.77:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 2ms, Average = 1ms
```

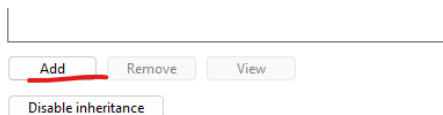
4. On the sharing computer, go to the folder you want to share, right-click it, and select **Properties**. Inside Properties, go to the **Sharing** tab → click **Advanced Sharing** → check the “**Share this folder**” box.



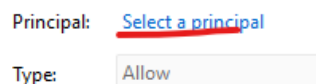
5. While still in Properties, switch to the **Security** tab and click on **Advanced**.



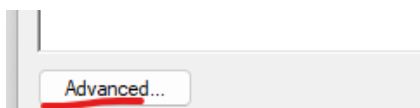
6. In the Advanced Security Settings window, click on **Add** to add a new permission entry.



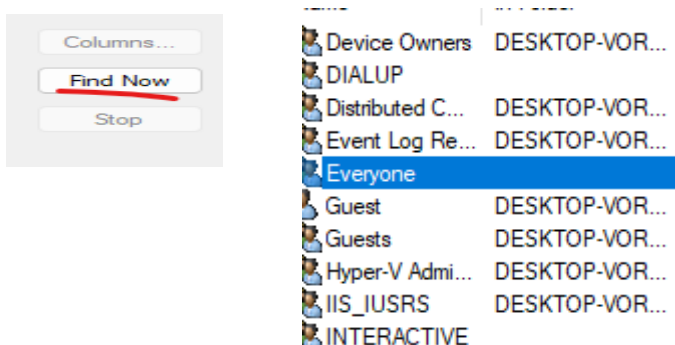
7. In the new entry tab, click on **Select a principal**.



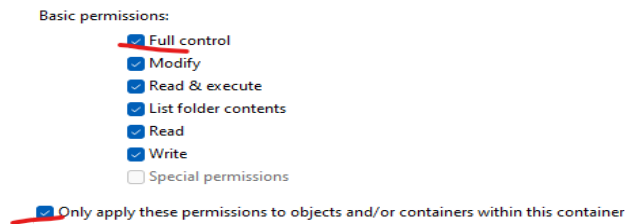
8. In the Select User or Group window, click on **Advanced**.



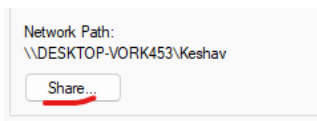
9. Click **Find Now**, scroll through the list and select **Everyone**.



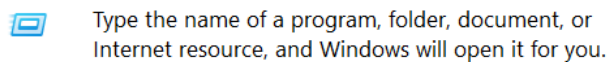
10. After selecting Everyone, set the appropriate permissions (e.g., Read or Full Control), then confirm and close the tab.



11. Back in the main Sharing tab, click the **Share** button to finalize folder sharing.



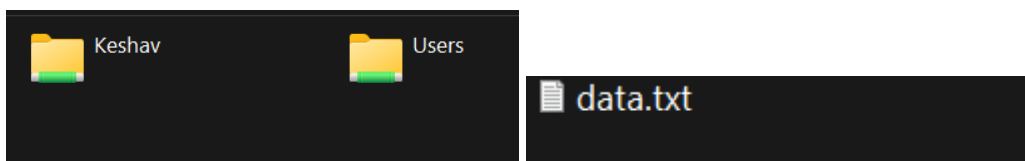
12. On another computer, press **Win + R**, type **\\<IP address>** of the sharing computer, and press Enter to access the shared folder..



Open: \\192.168.1.82

Observation:

- The folder was successfully accessed from another system using the IP address.
- Data transfer was completed without external storage devices.



Conclusion:

Successfully shared data over a LAN network and understood the practical working of LAN-based file sharing.

Objective:

SIMULATION OF MESSAGE TRANSFER IN A COMPUTER NETWORK USING CISCO PACKET TRACER

Apparatus / Requirements:

Cisco Packet Tracer software

One switch (Layer 2)

Multiple endpoint devices (e.g., PCs or laptops)

Ethernet cables for connecting devices to the switch

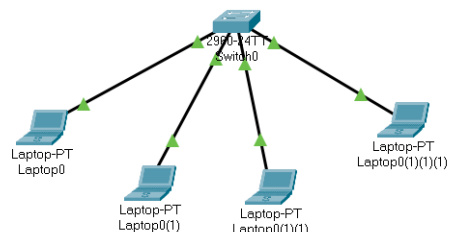
Proper IP addressing scheme for static IP configuration

Theory:

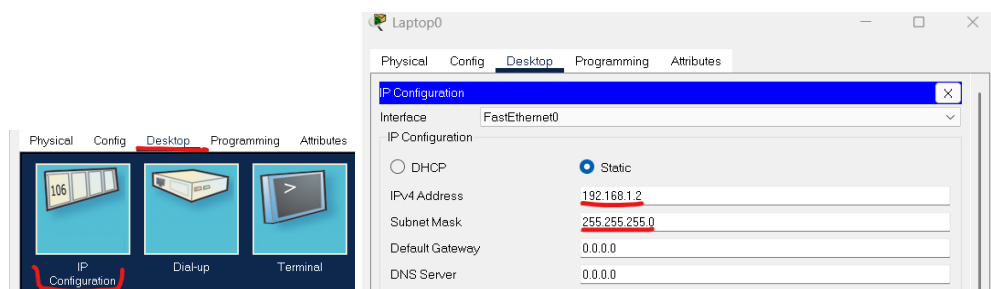
In computer networking, message transfer refers to the process of sending data from one device to another over a network. A basic network typically includes endpoint devices (such as computers), a central device like a switch to interconnect them, and proper IP addressing to enable communication. Cisco Packet Tracer is a simulation tool that allows users to design, configure, and test network setups virtually. By assigning static IP addresses to each device and connecting them through a switch, users can simulate data exchange. The Simple PDU (Protocol Data Unit) tool in Packet Tracer helps visualize the flow of packets between devices.

Procedure:

1. The Cisco Packet Tracer software was launched, and a simple network was designed using a switch and multiple endpoint devices.



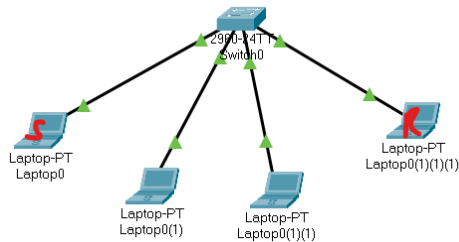
2. Each endpoint device was configured with a unique static IP address by accessing the **Desktop > IP Configuration** menu. The IPs were assigned sequentially by incrementing the last octet.



3. After setting the IP addresses, the **Simple PDU** tool was used to simulate message transfer between devices.



4. The first click on a device set it as the sender, and the second click selected the receiver. The selected devices successfully initiated a simulated message exchange.



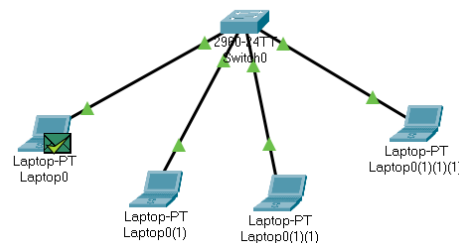
5. The simulation mode was started by clicking the simulation icon, and the **Play** button initiated the packet flow visualization.



6. The simulation successfully showed the message being transferred between the selected devices, confirming proper connectivity and configuration.

Observation:

The message was successfully transferred between the selected devices through the switch. All devices responded correctly, indicating proper IP configuration and network setup. The data flow during transmission is clearly shown in the attached picture.



Conclusion:

The simulation demonstrated successful message transfer in a basic network setup using Cisco Packet Tracer. It confirmed that with correct IP configuration and proper connectivity, devices can effectively communicate, and the data flow can be visually analyzed through simulation mode.

Objective:

TO SIMULATE THE CREATION AND IMPLEMENTATION OF VLANS IN A COMPUTER NETWORK USING CISCO PACKET TRACER.

Apparatus / Requirements:

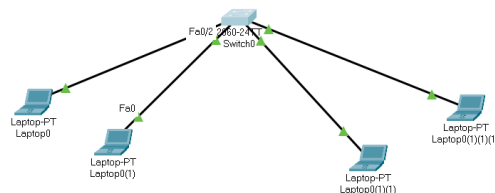
1. Cisco Packet Tracer software
2. One or more Cisco switches
3. Multiple endpoint devices (e.g., PCs or laptops)
4. Ethernet cables for connections
5. Basic knowledge of CLI for switch configuration
6. IP addressing plan for assigning static IPs

Theory:

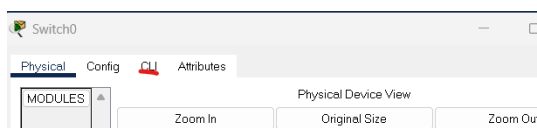
A VLAN (Virtual Local Area Network) is a logical subdivision of a physical network that allows devices to be grouped together based on function, department, or application, regardless of their physical location. VLANs improve network efficiency, security, and manageability by limiting broadcast domains and isolating traffic between groups.

Procedure:

1. First, we set up the network with 4 endpoint devices and 1 switch. Static IP addresses were configured on all devices as described in “Objective 2” of this practical file.



2. The switch was selected, and the **CLI** tab was opened to enter configuration commands.



3. The following commands were executed to create two VLANs on the switch.

```
Switch>enable
Switch#conf terminal
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 2
Switch(config-vlan)#name office
Switch(config-vlan)#exit
Switch(config)#vlan 3
Switch(config-vlan)#home
```

4. After creating the VLANs, each device was assigned to either the **Office** or **Home** VLAN by configuring the corresponding switch ports.

```
Switch(config-vlan)#exit
Switch(config)#interface fastEthernet 0/1
Switch(config-if)#switchport access vlan 2
Switch(config-if)#exit
Switch(config)#switchport access vlan 2
Switch(config)#
^
% Invalid input detected at '^' marker.

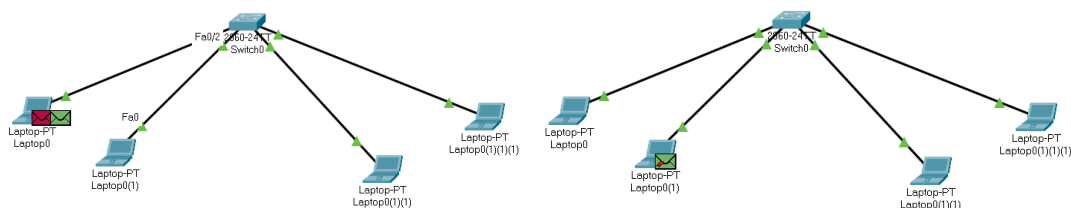
Switch(config)#interfact fastEthernet 0/2
Switch(config)#
^
% Invalid input detected at '^' marker.

Switch(config)#interface fastEthernet 0/2
Switch(config-if)#switchport access vlan 2
Switch(config-if)#exit
Switch(config)#interface fastEthernet 0/3
Switch(config-if)#switchport access vlan 3
Switch(config-if)#exit
Switch(config)#interface fastEthernet 0/4
Switch(config-if)#switchport access vlan 3
Switch(config-if)#exit
Switch(config)#
```

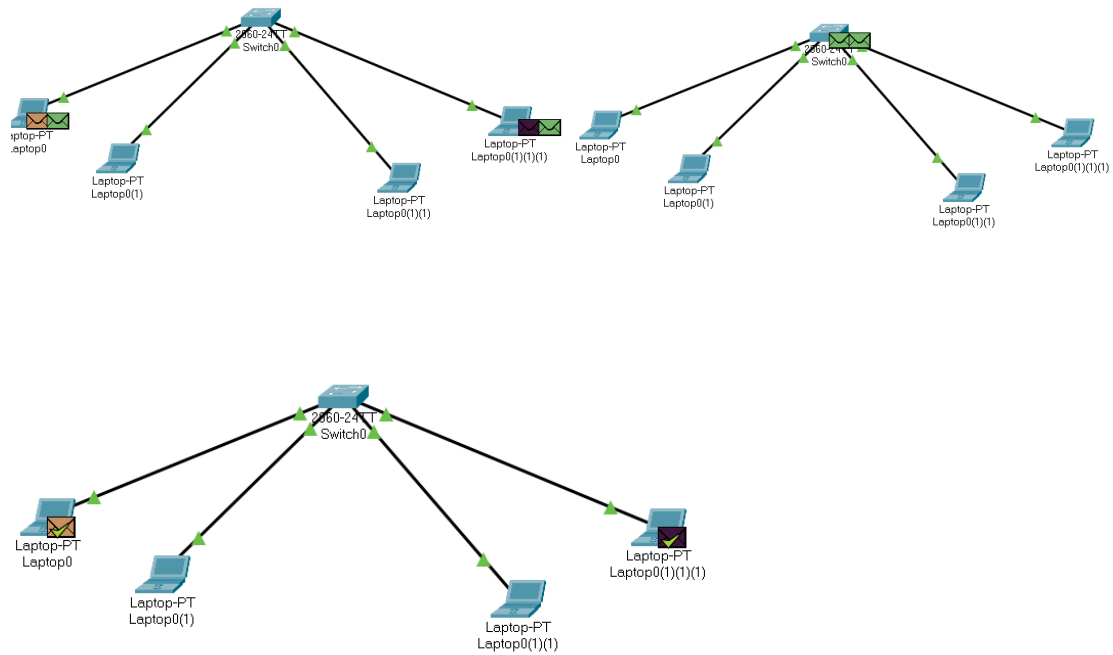
5. The VLAN setup for **Office** and **Home** was successfully completed. **FastEthernet 0/1** and **0/2** were assigned to the **Office** VLAN, while **FastEthernet 0/3** and **0/4** were assigned to the **Home** VLAN.

Observation:

1. First, we tried to send a Simple PDU from **FastEthernet 0/1** to **0/4**, but encountered an error, as shown in the following image. This occurred because **0/1** and **0/2** are in the same VLAN (Office), and **0/3** and **0/4** are in a different VLAN (Home). Since **0/1** and **0/4** belong to different VLANs, the switch forwarded the packet to **0/2** (same VLAN as the source), but the intended destination was **0/4**, resulting in a failed delivery



2. Next, we sent a Simple PDU from **FastEthernet 0/1** to **0/2** and from **0/4** to **0/3**. No errors were encountered, as the message transmission occurred within the same VLAN in both cases. This successful communication is shown in the images below.



Conclusion:

The simulation demonstrated that VLANs effectively segment a network by restricting communication between devices on different VLANs. Devices within the same VLAN communicate successfully, while attempts to send messages across VLANs without routing resulted in errors. This confirms that VLANs enhance network organization, security, and traffic management by isolating broadcast domains.

Objective:

SIMULATION OF STATIC NETWORKING BETWEEN TWO NETWORKS USING CISCO PACKET TRACER

Apparatus / Requirements:

- Cisco Packet Tracer software
- Two routers
- Two switches (one per network)
- Multiple endpoint devices (e.g., PCs or laptops) in each network
- Ethernet cables for connecting devices to switches and routers
- Proper static IP addressing scheme for both networks
- Static route configuration on each router

Theory:

Static networking between two networks involves connecting separate LANs (Local Area Networks) via routers, with manually configured routing tables. Each network contains its own group of endpoint devices connected through a switch. The routers enable inter-network communication by forwarding packets across networks based on static routing entries. In this setup, each PC is assigned a unique static IP address within its own network, and the routers are configured with static routes pointing to each other's networks. Cisco Packet Tracer allows for visual design, configuration, and simulation of such network topologies. Static routing requires manual configuration of the network path in each router's routing table. Unlike dynamic routing, which automatically learns routes, static routing is simple, secure, and useful in smaller networks or lab environments where topology changes are minimal. Once the static IPs are assigned and routing is configured correctly, devices from different networks can communicate seamlessly through the routers. Cisco Packet Tracer's simulation mode can then be used to verify the connectivity and visualize the flow of packets between the two separate networks.

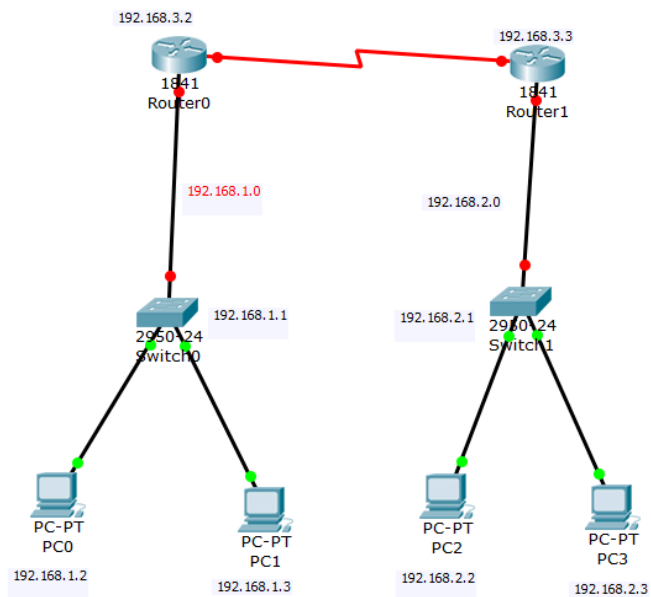
Procedure:

1. IP Configuration of Devices:

Begin by configuring the IP addresses of all devices according to the given network topology. Ensure each endpoint device (e.g., PCs) has:

- A correct **IP address**
- A matching **subnet mask**
- The appropriate **default gateway** as shown in **Objective 1**.

As shown in the figure



2.Router Configuration:

After assigning IPs to endpoints, proceed to configure the routers using the following commands. This set up the router interfaces and defined static routes between the networks.

For Router0:

```
Router> enable
Router# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
! Configure FastEthernet0/0
Router(config)# interface FastEthernet0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
Router(config-if)# exit
! Configure Serial0/1/0
Router(config)# interface Serial0/1/0
Router(config-if)# ip address 192.168.3.2 255.255.255.0
Router(config-if)# no shutdown
%LINK-5-CHANGED: Interface Serial0/1/0, changed state to down
%LINK-5-CHANGED: Interface Serial0/1/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial0/1/0, changed state to up
Router(config-if)# exit
! Add static route to network 192.168.2.0 via next hop 192.168.3.3
Router(config)# ip route 192.168.2.0 255.255.255.0 192.168.3.3
Router(config)#
```

For Router1:

```
Router> enable
Router# configure terminal
```

```

Enter configuration commands, one per line. End with CNTL/Z.
! Set up FastEthernet0/0 with static IP
Router(config)# interface FastEthernet0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
Router(config-if)# exit
! Set up Serial0/1/0 with static IP
Router(config)# interface Serial0/1/0
Router(config-if)# ip address 192.168.3.2 255.255.255.0
Router(config-if)# no shutdown
%LINK-5-CHANGED: Interface Serial0/1/0, changed state to down
%LINK-5-CHANGED: Interface Serial0/1/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial0/1/0, changed state to up
Router(config-if)# exit
! Configure static route to remote network via next hop
Router(config)# ip route 192.168.2.0 255.255.255.0 192.168.3.3
Router(config)#

```

3. At this point, both routers were fully configured. You could verify the setup by sending a **Simple PDU** between endpoints across different networks to confirm connectivity.

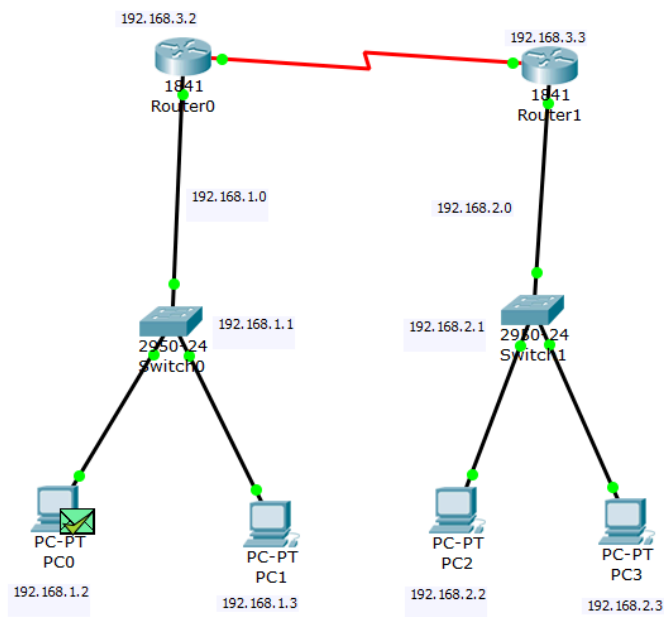
OBSERVATION

To verify end-to-end connectivity, we sent a **Simple PDU** from **PC 1.2 (IP: 192.168.1.2)** to **PC 2.2 (IP: 192.168.2.2)**. The packet successfully moved:

- From PC 1.2 to its connected **switch**,
- From the switch to **Router0** via the FastEthernet interface,
- Across the **Serial connection** to **Router1**,
- Then out through Router1's FastEthernet port to the second switch,
- And finally reached PC 2.2.

Upon arrival at PC 2.2, an **acknowledgment (ACK)** packet was automatically generated and sent back to PC 1.2 following the reverse route. The complete message exchange was marked as **successful** in the Packet Tracer simulation, as shown in the result figure.

This successful test confirmed that the static routing configuration was correct and that inter-network communication was functioning as intended.



Conclusion:

The simulation demonstrated that static routing effectively enables communication between devices on different networks. By manually configuring IP addresses and static routes on each router, devices in separate subnets were able to exchange messages successfully. The successful transmission and return of acknowledgment packets confirmed proper router configuration and route setup. This highlights that static routing provides a reliable method for directing network traffic in small or controlled environments, ensuring precise path control and predictable behavior without the complexity of dynamic protocols.

Objective:

SIMULATION OF DYNAMIC NETWORKING BETWEEN TWO NETWORKS
USING CISCO PACKET TRACER

Apparatus / Requirements:

- Cisco Packet Tracer software
- Two routers
- Two switches (one per network)
- Multiple endpoint devices (e.g., PCs or laptops) in each network
- Ethernet cables for connecting devices to switches and routers
- Proper dynamic IP addressing scheme for both networks
- Static route configuration on each router

Theory:

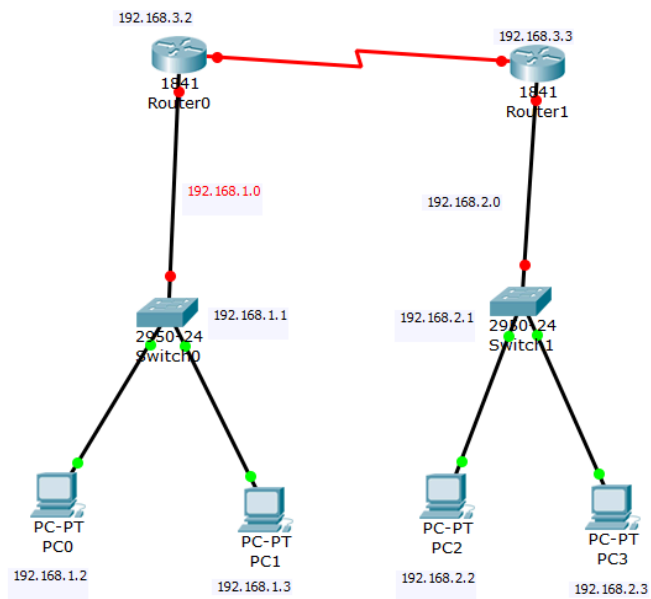
In computer networking, **routing** is the process of selecting paths for traffic in a network. There are two main types of routing: **static routing** and **dynamic routing**. In **dynamic routing**, routers automatically learn and update routes to other networks using routing protocols. Dynamic routing protocols such as **RIP (Routing Information Protocol)**, **OSPF (Open Shortest Path First)**, and **EIGRP (Enhanced Interior Gateway Routing Protocol)** enable routers to exchange routing information with one another. This allows routers to dynamically adjust to changes in the network topology, such as link failures or the addition of new networks, without manual intervention. For example, **RIP** is a distance-vector routing protocol that uses hop count as a metric. It periodically broadcasts the routing table to neighboring routers, allowing all routers to gradually build up a complete view of the network. RIP is simple and suitable for small to medium-sized networks. The advantage of dynamic routing over static routing is that it reduces administrative overhead and enhances scalability. While static routing is prone to human error and is difficult to maintain in large networks, dynamic routing responds to topology changes automatically, improving the reliability and adaptability of the network.

In this lab, we simulate dynamic routing using RIP to establish communication between different networks, allowing endpoint devices in separate subnets to exchange data seamlessly.

Procedure:

1. IP Configuration of Devices:

Began by assigning static IP addresses and default gateways to all endpoint devices (e.g., PCs), following the network diagram. Ensure that each device belongs to the correct subnet and was properly connected to its respective switch.



2.Router Configuration:

Configured router interfaces with appropriate IP addresses and enabled them using the `no shutdown` command. This established basic network connectivity on both ends.

For Router0

```
Router> enable
Router# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
```

! Configure FastEthernet0/0

```
Router(config)# interface FastEthernet0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
```

! Configure Serial0/1/0

```
Router(config)# interface Serial0/1/0
Router(config-if)# ip address 192.168.3.2 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
```

! Enable RIP and advertise networks

```
Router(config)# router rip
Router(config-router)# version 2
Router(config-router)# no auto-summary
Router(config-router)# network 192.168.1.0
Router(config-router)# network 192.168.2.0
Router(config-router)# network 192.168.3.0
Router(config-router)# exit
```

For Router1

```
Router> enable
Router# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
! Configure FastEthernet0/0
Router(config)# interface FastEthernet0/0
Router(config-if)# ip address 192.168.2.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
! Configure Serial0/1/0
Router(config)# interface Serial0/1/0
Router(config-if)# ip address 192.168.3.3 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
! Enable RIP and advertise networks
Router(config)# router rip
Router(config-router)# version 2
Router(config-router)# no auto-summary
Router(config-router)# network 192.168.2.0
Router(config-router)# network 192.168.1.0
Router(config-router)# network 192.168.3.0
Router(config-router)# exit
```

3. We can see the RIP being added

Network Address	Network Address
192.168.1.0	192.168.1.0
192.168.2.0	192.168.2.0
192.168.3.0	192.168.3.0

Router0

Router1

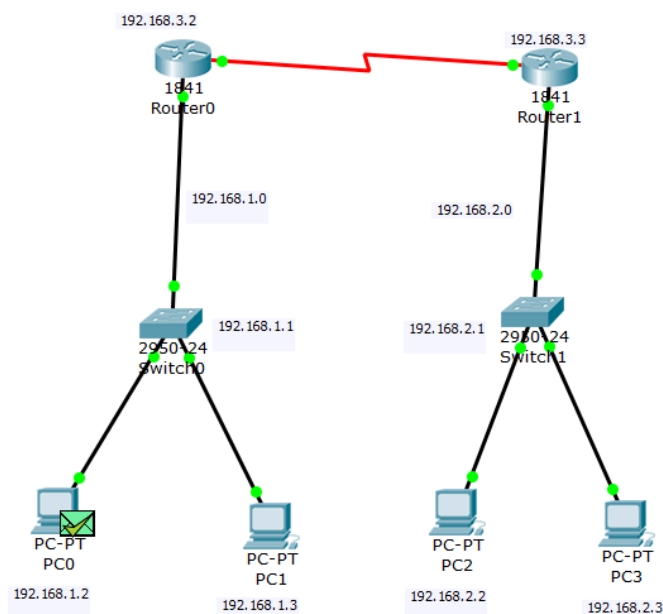
Verification:

We could verify that routers have dynamically learned routes to each other's networks. Then, perform ping tests or send a **Simple PDU** between endpoints across different networks to confirm connectivity.

OBSERVATION

A **Simple PDU** was sent from **PC 1.2 (192.168.1.2)** to **PC 2.2 (192.168.2.2)**. The packet traveled:

- From PC 1.2 to its local switch,
- To Router0 via FastEthernet0/0,
- Across the Serial link to Router1,
- To the second switch via FastEthernet0/0,
- And successfully reached PC 2.2.



Conclusion:

The simulation demonstrated that dynamic routing protocols such as RIP can automatically manage routing information between routers without manual configuration of static routes. By advertising directly connected networks, routers dynamically learned paths to remote networks. The successful exchange of data between devices across different subnets confirmed proper dynamic route propagation. This approach simplifies network management, enhances scalability, and reduces configuration overhead in larger or frequently changing networks.

Objective:

STUDY OF NETWORK PACKET USING WIRESHARK

Apparatus / Requirements:

- Wireshark

Theory

Wireshark is an open-source network packet analyzer used to monitor and examine data packets transmitted across a network. It captures network traffic from a live connection or reads saved capture files, decoding the packets according to the appropriate protocol specifications.

It supports hundreds of protocols (e.g., TCP, UDP, HTTP, FTP, ICMP, DNS) and works on various operating systems. Wireshark enables users to:

- Understand protocol behavior and packet structure.
- Troubleshoot network latency, dropped packets, or misconfigurations.
- Detecting suspicious or malicious network activity.
- Learn networking concepts through visual analysis.

It operates in promiscuous mode, allowing the capture of all packets on the network segment, not just those addressed to the capturing device. Packets are displayed in a layered view showing frame information, protocol headers, and payload data.

Procedure:

- The desired network interface (e.g., Wi-Fi or Ethernet) was selected.
- Start was clicked to begin capturing network packets.
- (Optional) Display filters were applied to focus on specific protocols or ports.
- Individual packets were selected to view detailed protocol information.
- Stop was clicked once enough data was captured.
- Packet details were analyzed for source, destination, protocol, and payload.
- The capture file was saved for documentation or further study.

Observation:

For udp:

```
▼ Frame 801: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface \Device\NPF_{8C5B8DB7-D9E3-4778-8F14-AE8341BFCB87}, id 0
  Section number: 1
  ▶ Interface id: 0 (\Device\NPF_{8C5B8DB7-D9E3-4778-8F14-AE8341BFCB87})
    Encapsulation type: Ethernet (1)
    Arrival Time: Aug 10, 2025 19:56:47.514680000 Nepal Standard Time
    UTC Arrival Time: Aug 10, 2025 14:11:47.514680000 UTC
    Epoch Arrival Time: 1754835107.514680000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.020260000 seconds]
    [Time delta from previous displayed frame: 0.692072000 seconds]
    [Time since reference or first frame: 5.334260000 seconds]
    Frame Number: 801
    Frame Length: 86 bytes (688 bits)
    Capture Length: 86 bytes (688 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:udp:rtcp]
    [Coloring Rule Name: UDP]
    [Coloring Rule String: udp]
  ▼ Ethernet II, Src: TaicangT&WEL_4a:f7:40 (5c:8c:30:4a:f7:40), Dst: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3)
    ▶ Destination: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3)
    ▶ Source: TaicangT&WEL_4a:f7:40 (5c:8c:30:4a:f7:40)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  ▼ Internet Protocol Version 4, Src: 35.207.227.110, Dst: 192.168.1.94
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0x60 (DSCP: CS3, ECN: Not-ECT)
    Total Length: 72
    Identification: 0x00b4 (180)
    ▶ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 57
    Protocol: UDP (17)
    Header Checksum: 0x774d [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 35.207.227.110
```

```
Source Address: 35.207.227.110
Destination Address: 192.168.1.94
[Stream index: 0]
▼ User Datagram Protocol, Src Port: 50008, Dst Port: 54567
  Source Port: 50008
  Destination Port: 54567
  Length: 52
  Checksum: 0x4190 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 2]
  [Stream Packet Number: 99]
  ▶ [Timestamps]
  UDP payload (44 bytes)
  ▼ Real-time Transport Control Protocol (Application specific)
    10. .... = Version: RFC 1889 Version (2)
    ..0. .... = Padding: False
    ...0 0001 = Subtype: 1
    Packet type: Application specific (204)
    Length: 5 (24 bytes)
    Identifier: 0x0000da2d (55853)
    ▶ Application name is not a string
    Application specific data: f5b2aa1ed1dc5ffd057063b5
    [RTCP frame length check: OK - 24 bytes]
```

```
0000 c0 bf be 2d 05 d3 5c 8c 30 4a f7 40 08 00 45 60 ..... \ 0! @ E'
```

For tcp:

```
▼ Frame 110129: 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface \Device\NPF_{8C5B8DB7-D9E3-4778-8F14-AE8341BFCB87}, id 0
  Section number: 1
  ▶ Interface id: 0 (\Device\NPF_{8C5B8DB7-D9E3-4778-8F14-AE8341BFCB87})
    Encapsulation type: Ethernet (1)
    Arrival Time: Aug 10, 2025 20:04:44.602870000 Nepal Standard Time
    UTC Arrival Time: Aug 10, 2025 14:19:44.602870000 UTC
    Epoch Arrival Time: 1754835584.602870000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.007291000 seconds]
    [Time delta from previous displayed frame: 0.269452000 seconds]
    [Time since reference or first frame: 482.422450000 seconds]
    Frame Number: 110129
    Frame Length: 78 bytes (624 bits)
    Capture Length: 78 bytes (624 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp:tls]
    [Coloring Rule Name: TCP]
    [Coloring Rule String: tcp]
  ▼ Ethernet II, Src: TaicangT&WEL_4a:f7:40 (5c:8c:30:4a:f7:40), Dst: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3)
    ▶ Destination: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3)
    ▶ Source: TaicangT&WEL_4a:f7:40 (5c:8c:30:4a:f7:40)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  ▼ Internet Protocol Version 4, Src: 108.158.61.128, Dst: 192.168.1.94
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0xb8 (DSCP: EF, ECN: Not-ECT)
    Total Length: 64
    Identification: 0x953d (38205)
    ▶ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 247
    Protocol: TCP (6)
    Header Checksum: 0xc19d [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 108.158.61.128
```

```

Source Address: 108.158.61.128
Destination Address: 192.168.1.94
[Stream index: 2]
▼ Transmission Control Protocol, Src Port: 443, Dst Port: 12428, Seq: 11165, Ack: 225, Len: 24
  Source Port: 443
  Destination Port: 12428
  [Stream index: 3]
  [Stream Packet Number: 140]
  ▶ [Conversation completeness: Incomplete (12)]
  [TCP Segment Len: 24]
  Sequence Number: 11165 (relative sequence number)
  Sequence Number (raw): 141213262
  [Next Sequence Number: 11189 (relative sequence number)]
  Acknowledgment Number: 225 (relative ack number)
  Acknowledgment number (raw): 3300809488
  0101 .... = Header Length: 20 bytes (5)
  ▶ Flags: 0x018 (PSH, ACK)
  Window: 473
  [Calculated window size: 473]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0x9f44 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ [Timestamps]
  ▶ [SEQ/ACK analysis]
  TCP payload (24 bytes)
▼ Transport Layer Security
  ▶ TLSv1.2 Record Layer: Application Data Protocol: Hypertext Transfer Protocol

```

For ip:

```

▼ Frame 164131: 261 bytes on wire (2088 bits), 261 bytes captured (2088 bits) on interface \Device\NPF_{8C5B8DB7-D9E3-4778-8F14-AE8341BFCB87}, id 0
  Section number: 1
  ▶ Interface id: 0 (\Device\NPF_{8C5B8DB7-D9E3-4778-8F14-AE8341BFCB87})
  Encapsulation type: Ethernet (1)
  Arrival Time: Aug 10, 2025 20:07:50.775361000 Nepal Standard Time
  UTC Arrival Time: Aug 10, 2025 14:22:50.775361000 UTC
  Epoch Arrival Time: 1754835770.775361000
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 0.007456000 seconds]
  [Time delta from previous displayed frame: 0.018065000 seconds]
  [Time since reference or first frame: 668.594941000 seconds]
  Frame Number: 164131
  Frame Length: 261 bytes (2088 bits)
  Capture Length: 261 bytes (2088 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:udp:data]
  [Coloring Rule Name: UDP]
  [Coloring Rule String: udp]
▼ Ethernet II, Src: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3), Dst: TaicangT&WE1_4a:f7:40 (5c:8c:30:4a:f7:40)
  ▶ Destination: TaicangT&WE1_4a:f7:40 (5c:8c:30:4a:f7:40)
  ▶ Source: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3)
  Type: IPv4 (0x0800)
  [Stream index: 0]
▼ Internet Protocol Version 4, Src: 192.168.1.94, Dst: 35.207.227.110
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 247
  Identification: 0x808f (32911)
  ▶ 000. .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 128
  Protocol: UDP (17)
  Header Checksum: 0xf022 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.1.94

```

```

  ▶ Source: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3)
  Type: IPv4 (0x0800)
  [Stream index: 0]
▼ Internet Protocol Version 4, Src: 192.168.1.94, Dst: 35.207.227.110
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 247
  Identification: 0x808f (32911)
  ▶ 000. .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 128
  Protocol: UDP (17)
  Header Checksum: 0xf022 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.1.94
  Destination Address: 35.207.227.110
  [Stream index: 0]
▼ User Datagram Protocol, Src Port: 53674, Dst Port: 50008
  Source Port: 53674
  Destination Port: 50008
  Length: 227
  Checksum: 0xb9cc [unverified]
  [Checksum Status: Unverified]
  [Stream index: 171]
  [Stream Packet Number: 205]
  ▶ [Timestamps]
  UDP payload (219 bytes)
▼ Data (219 bytes)
  Data [...]: 907859c43caf81ea000d929bede0002a62321516be17a480054e2e5a452b604768ec85b9bb177abbad3c1a6cdc2d6e0ee4c731205b3205fb9b148ec8eda107b56c5202ba7ee8e9f63...
  [Length: 219]

```

For arp:

```
▼ Frame 144728: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{8C5B8DB7-D9E3-4778-8F14-AE8341BFCB87}, id 0
  Section number: 1
  ▶ Interface id: 0 (\Device\NPF_{8C5B8DB7-D9E3-4778-8F14-AE8341BFCB87})
  Encapsulation type: Ethernet (1)
  Arrival Time: Aug 10, 2025 20:06:37.526110000 Nepal Standard Time
  UTC Arrival Time: Aug 10, 2025 14:21:37.526110000 UTC
  Epoch Arrival Time: 1754835697.526110000
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 0.000016000 seconds]
  [Time delta from previous displayed frame: 0.000016000 seconds]
  [Time since reference or first frame: 595.345690000 seconds]
  Frame Number: 144728
  Frame Length: 42 bytes (336 bits)
  Capture Length: 42 bytes (336 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:arp]
  [Coloring Rule Name: ARP]
  [Coloring Rule String: arp]
▼ Ethernet II, Src: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3), Dst: TaicangT&WE1_4a:f7:40 (5c:8c:30:4a:f7:40)
  ▶ Destination: TaicangT&WE1_4a:f7:40 (5c:8c:30:4a:f7:40)
  ▶ Source: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3)
  Type: ARP (0x0806)
  [Stream index: 0]
▼ Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: AzureWaveTec_2d:05:d3 (c0:bf:be:2d:05:d3)
  Sender IP address: 192.168.1.94
  Target MAC address: TaicangT&WE1_4a:f7:40 (5c:8c:30:4a:f7:40)
  Target IP address: 192.168.1.254
```

Conclusion:

The study of network packets using Wireshark provided practical insight into how data is transmitted and received across a network. By capturing and analyzing packets, the structure of various network protocols was understood, including their headers and payloads. This helped in identifying communication patterns, protocol behavior, and possible anomalies in the network. Overall, the experiment demonstrated Wireshark's effectiveness as a powerful tool for learning, troubleshooting, and monitoring network traffic.